INSTITUTO MAUÁ DE TECNOLOGIA

MAUÁ

ECM253 – Linguagens Formais, Autômatos e Compiladores

Atividade

Analisador Recursivo Descendente para a linguagem Jaca

Parte I

Marco Furlan

Setembro de 2026

1 Introdução

A linguagem Jaca (*Just Another Compiler Attempt*) é uma linguagem simples cuja gramática está definida a seguir:

```
(*
    Gramática para a linguagem Jaca ("Just Another Compiler Attempt")
    © Marco AFS - 2025
*)
program = statement_list,EOF;
statement_list = statement,{statement};
statement = assign_stmt
          | if_stmt
          | while_stmt
          | block_stmt
          | null_stmt;
assign_stmt = ID,"=",expression,";";
if_stmt = "if","(",expression,")","then",block_stmt,["else",block_stmt];
while_stmt = "while","(",expression,")",block_stmt;
block_stmt = "{",statement_list,"}";
null_stmt = ";";
expression = simple_expr,[REL_OPERATOR,simple_expr];
simple_expr = term,{ADD_OPERATOR,term};
term = factor,{MUL_OPERATOR,factor};
factor = ID
       | NUMBER
       | "(",expression,")";
(* Terminais -> gramática léxica - descrição em ER *)
ID = "[A-Za-z_][A-Za-z_0-9]*"
NUMBER = "[0-9]+(\.[0-9]+)?([Ee][+-]?[0-9]+)?"
ADD_OPERATOR = "[+-]"
```

```
MUL_OPERATOR = "[\*\/\%]"
REL_OPERATOR = "(==|!=|<|>|<=|>=)"
EOF = -1 (* marca correspondente ao final da entrada (-1) *)
```

2 Projeto

Nesta parte você deverá **elaborar** e **testar** o **analisador léxico** para a **linguagem Jaca**. Para tanto:

- **Baixar** e **descompactar** o **arquivo** `Analisador Léxico para Jaca.zip` que está no **Canvas** da disciplina.
- A partir da **gramática** de **Jaca**, adicionar novas **classes herdeiras** de `Token` para representar os **terminais** da **linguagem** no diretório `src/main/java/compiler/scanner`.
- **Adicionar** em `src/main/java/compiler/scanner` **nomes de constantes** para identificar as **marcas** que serão geradas.
- Criar **macros** de **expressões regulares** no arquivo `src/main/jflex` para representar as expressões regulares mais complexas.
- Criar **expressões regulares** (das macros ou diretamente) e **ações léxicas** que deverão **retornar** marcas (**objetos** de `Token` ou de **classes derivadas** dela) **adequadas para** cada **categoria** de **marca** elaborada.
- **Compilar** e **executar**. O **programa principal** está **preparado** para **exibir** as **marcas retornadas** (verifique).
- **Alterar** o **arquivo** de **testes** (`teste.input`) para **reconhecer todos** os **terminais válidos** de Jaca. **Testar terminais inválidos**, também.

3 O que fazer?

- Reunir em equipe de até 4 pessoas.
- Implementar o projeto durante a aula.
- Apresentar o projeto ao professor, testando-o. O professor anotará a presença de todos da equipe e avaliará o projeto.